SKILL WEEK-9

SESSION-17

Task-1: To Implement how to List Active Jobs, Display Status Information, Retrieve Job [Ds, Format Output, Update State Information, Test Job Listings].

Source code:

```
  GNU nano 8.7.1                    job_list.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <signal.h>

#define MAX_JOBS 10

typedef struct {
    int job_id;
    pid_t pid;
    char command[50];
    char state[20];
} Job;

Job jobs[MAX_JOBS];
int job_count = 0;

void add_job(pid_t pid, const char *command)
{
    if (job_count < MAX_JOBS) {
        jobs[job_count].job_id = job_count + 1;
        jobs[job_count].pid = pid;

        snprintf(jobs[job_count].command,
                 sizeof(jobs[job_count].command),
                 "%s", command);

        snprintf(jobs[job_count].state,
                 sizeof(jobs[job_count].state),
                 "Running");

        job_count++;
    }
}

void update_jobs()
{
    for (int i = 0; i < job_count; i++) {
        int status;
        pid_t result = waitpid(jobs[i].pid, &status, WNOHANG);

        if (result == jobs[i].pid) {
```

```
int main()
{
    printf("Job Management System Started\n");

    pid_t pid1 = fork();

    if (pid1 == 0) {
        sleep(10);
        return 0;
    }

    add_job(pid1, "sleep 10");

    pid_t pid2 = fork();

    if (pid2 == 0) {
        sleep(5);
        return 0;
    }

    add_job(pid2, "sleep 5");

    printf("\nJobs created successfully.\n");

    list_jobs();

    printf("\nWaiting for jobs to complete...\n");
    sleep(6);

    list_jobs();

    printf("\nFinal job status:\n");
    sleep(5);

    list_jobs();

    return 0;
}
```

Output:

```
Job Management System Started

Jobs created successfully.

------ Active Job List ------
Job ID  PID     Status          Command
-----------------------------------------
[1]     518     Running         sleep 10
[2]     519     Running         sleep 5
-----------------------------------------

Waiting for jobs to complete...

------ Active Job List ------
Job ID  PID     Status          Command
-----------------------------------------
[1]     518     Running         sleep 10
[2]     519     Completed       sleep 5
-----------------------------------------

Final job status:

------ Active Job List ------
Job ID  PID     Status          Command
-----------------------------------------
[1]     518     Completed       sleep 10
[2]     519     Completed       sleep 5
-----------------------------------------
```

Observation:

Initially, both jobs are displayed as **Running**. After the sleep 5 process finishes, its state changes to **Completed**. After the remaining process finishes, both jobs are shown as **Completed**.

Task-2: To design how to Identify Target Job, Transfer Terminal Control, Resume Processes, Wait for Completion, Update Job State, Test Foreground Switching.

Source code:

```
 GNU nano 8.7.1                foreground_switch.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <signal.h>
#include <stdlib.h>

int main()
{
    pid_t pid;
    int status;

    printf("Foreground Job Control Demo\n");
    printf("---------------------------\n");

    pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        /* Child process */
        printf("Child: Job started. PID = %d\n", getpid());

        /* Stop itself */
        printf("Child: Stopping myself...\n");
        raise(SIGSTOP);

        /* Execution continues after SIGCONT */
        printf("Child: Resumed in foreground.\n");
        printf("Child: Running for 5 seconds...\n");

        sleep(5);

        printf("Child: Job completed.\n");
        exit(0);
    }

    /* Parent process */
    printf("Parent: Target job identified.\n");
    printf("Parent: Job PID = %d\n", pid);
```

```c
    sleep(1);

    /* Detect stopped process */
    waitpid(pid, &status, WUNTRACED);

    if (WIFSTOPPED(status)) {
        printf("\n------ Job Status ------\n");
        printf("Job PID: %d\n", pid);
        printf("State: Stopped\n");
    }

    /* Transfer foreground control conceptually */
    printf("\nTransferring terminal control to Job [%d]...\n", pid);

    /* Resume stopped process */
    if (kill(pid, SIGCONT) == 0) {
        printf("Continue signal sent successfully.\n");
    }

    printf("Job [%d] resumed in foreground.\n", pid);

    /* Wait for foreground job */
    waitpid(pid, &status, 0);

    if (WIFEXITED(status)) {
        printf("\n------ Updated Job Status ------\n");
        printf("Job PID: %d\n", pid);
        printf("State: Completed\n");
        printf("Exit Status: %d\n", WEXITSTATUS(status));
    }

    printf("\nForeground switching test completed.\n");

    return 0;
}
```

Output:

```
Foreground Job Control Demo
---------------------------
Parent: Target Job identified.
Parent: Job PID = 663
Child: Job started. PID = 663
Child: Stopping myself...

------ Job Status ------
Job PID: 663
State: Stopped

Transferring terminal control to Job [663]...
Continue signal sent successfully.
Job [663] resumed in foreground.
Child: Resumed in foreground.
Child: Running for 5 seconds...
Child: Job completed.

------ Updated Job Status ------
Job PID: 663
State: Completed
Exit Status: 0

Foreground switching test completed.
```

Observation:

The child process initially enters the **Stopped** state. The parent identifies the target job and sends SIGCONT. The process resumes, executes in the foreground, and after completion its state is updated to **Completed**.

SESSION-18

Task-1: To Analyze Signal Delivery, Understand Signal Masks, Review Process Groups, Explore Terminal Signals, Test Signal Behavior.

Source code:

```
 GNU nano 8.7.1                    signal_demo.c *
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <stdlib.h>

void handle_signal(int sig)
{
    printf("\nSignal received: %d\n", sig);

    if (sig == SIGINT)
        printf("SIGINT (Ctrl+C) received.\n");

    if (sig == SIGTERM)
        printf("SIGTERM received.\n");

    fflush(stdout);
}

int main()
{
    sigset_t mask, oldmask;

    printf("Signal Analysis Program\n");
    printf("-----------------------\n");

    printf("Process ID     : %d\n", getpid());
    printf("Process Group : %d\n", getpgrp());

    /* Install signal handlers */
    signal(SIGINT, handle_signal);
    signal(SIGTERM, handle_signal);

    /* Create a signal mask */
    sigemptyset(&mask);
    sigaddset(&mask, SIGINT);

    printf("\nBlocking SIGINT for 10 seconds...\n");

    /* Block SIGINT */
    sigprocmask(SIG_BLOCK, &mask, &oldmask);
```

```
    /* Block SIGINT */
    sigprocmask(SIG_BLOCK, &mask, &oldmask);

    printf("Try pressing Ctrl+C now.\n");
    printf("SIGINT is currently blocked.\n");

    sleep(10);

    printf("\nUnblocking SIGINT...\n");

    /* Restore previous signal mask */
    sigprocmask(SIG_SETMASK, &oldmask, NULL);

    printf("SIGINT is now unblocked.\n");
    printf("Press Ctrl+C to test signal delivery.\n");

    printf("\nWaiting for signals...\n");

    while (1)
    {
        pause();
    }

    return 0;
}
```

Output:

```
Signal Analysis Program
-----------------------
Process ID     : 792
Process Group : 792

Blocking SIGINT for 10 seconds...
Try pressing Ctrl+C now.
SIGINT is currently blocked.

Unblocking SIGINT...
SIGINT is now unblocked.
Press Ctrl+C to test signal delivery.

Waiting for signals...
^C
Signal received: 2
SIGINT (Ctrl+C) received.
```

Observation:

The experiment successfully demonstrates **signal delivery, signal masking, process groups, terminal-generated signals, and signal behavior** in Linux. It also shows how these mechanisms are fundamental to implementing foreground and background job control in a shell.

Task-2: To implement how to resume Stopped Jobs, Send Continue Signals, Update Job Status, Maintain Job Table, Verifr Execution, Test Background Recovery.

Source code:

```
 GNU nano 8.7.1                  job_recovery.c *
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <stdlib.h>

#define MAX_JOBS 10

typedef struct {
    int job_id;
    pid_t pid;
    char command[50];
    char state[20];
} Job;

Job job_table[MAX_JOBS];
int job_count = 0;

/* Add a job to the job table */
void add_job(pid_t pid, const char *command)
{
    job_table[job_count].job_id = job_count + 1;
    job_table[job_count].pid = pid;

    snprintf(job_table[job_count].command,
             sizeof(job_table[job_count].command),
             "%s", command);

    snprintf(job_table[job_count].state,
             sizeof(job_table[job_count].state),
             "Stopped");

    job_count++;
}

/* Display job table */
void display_jobs()
{
    printf("\n------ Job Table ------\n");
    printf("Job ID\tPID\tState\t\tCommand\n");
    printf("-----------------------------------------\n");
```

```c
    display_jobs();

    /* Resume the stopped job */
    resume_job(1);

    display_jobs();

    /* Wait for job completion */
    printf("\nWaiting for background job...\n");

    waitpid(pid, &status, 0);

    /* Update final status */
    for (int i = 0; i < job_count; i++) {

        if (job_table[i].pid == pid) {

            if (WIFEXITED(status)) {
                snprintf(job_table[i].state,
                         sizeof(job_table[i].state),
                         "Completed");
            }
            else if (WIFSIGNALED(status)) {
                snprintf(job_table[i].state,
                         sizeof(job_table[i].state),
                         "Terminated");
            }
        }
    }

    printf("\nFinal Job Status:\n");
    display_jobs();

    printf("\nBackground recovery test completed.\n");

    return 0;
}
```

Output:

```
Background Job Recovery System
------------------------------
Parent: Job created with PID 900
Child: PID = 900
Child: Job started.
Child: Stopping myself...

Job [1] stopped.
Stopped signal: 19

------ Job Table ------
Job ID  PID     State           Command
---------------------------------------
[1]     900     Stopped         background-task
---------------------------------------

Target Job: [1]
PID: 900
Sending SIGCONT signal...
SIGCONT sent successfully.
Child: Job resumed successfully.
Job [1] resumed.
Child: Running in background...

------ Job Table ------
Job ID  PID     State           Command
---------------------------------------
[1]     900     Running         background-task
---------------------------------------

Waiting for background job...
Child: Background job completed.

Final Job Status:

------ Job Table ------
Job ID  PID     State           Command
---------------------------------------
[1]     900     Completed       background-task
---------------------------------------
```

Observation:

The stopped job is successfully detected and stored in the job table. The SIGCONT signal resumes the process, and its state changes from **Stopped → Running → Completed** after successful execution.